Reinforcement Learning for Language Models

MDPs · Q-Learning · Policy Gradients · RLHF · PPO · DPO · GRPO

Why reinforcement learning for NLP?

Pre-training objective

Predict the next token
Minimizes cross-entropy
on internet text

$\neq$

What we actually want

Helpful, harmless, honest
Follow instructions
Reason correctly

The alignment gap: "good" outputs can't always be expressed as a differentiable loss.

Human preference, correctness of code, safety — these are **rewards**, not gradients.

RL lets us optimize for **any reward signal**, even non-differentiable ones.

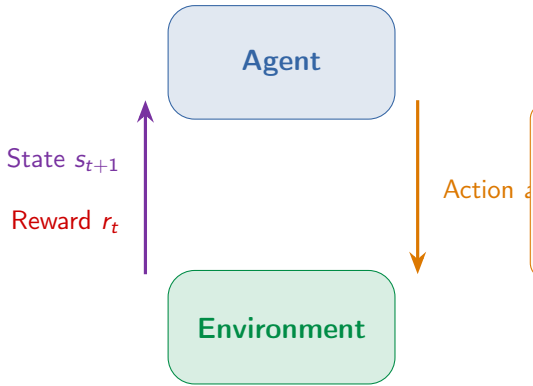
RL in modern LLMs: **RLHF** (alignment) · **GRPO** (reasoning) · **Code rewards** (pass@k) · **Tool use** (task success)

Part I

Classical RL Foundations

MDPs · Value Functions · Q-Learning · Policy Gradients

The RL framework: Markov Decision Process



Goal: find policy $\pi(a|s)$ that maximizes

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

$$\text{MDP} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

$\mathcal{S}$: states $\mathcal{A}$: actions $P(s'|s,a)$: transition $R(s,a)$: reward γ: discount factor

Value functions

State-value function $V^\pi(s)$

$$V^\pi(s) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \;\middle|\; s_t = s\right]$$

"How good is it to **be** in state s under policy π?"

Action-value function $Q^\pi(s, a)$

$$Q^\pi(s, a) = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \;\middle|\; s_t = s, a_t = a\right]$$

"How good is it to **take action** a in state s under policy π?"

Bellman equation (the recursive structure of value):

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \sum_{a'} \pi(a'|s') \, Q^\pi(s', a')$$

Value now = immediate reward + discounted future value. This recursion is the foundation of RL.

Optimal policy: $\pi^*(s) = \arg\max_a Q^*(s, a)$ where

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

Q-Learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{TD target}} - \underbrace{Q(s,a)}_{\text{current estimate}}\Big]$$

Model-free

No need to know $P(s'|s,a)$

Off-policy

Learns Q^* regardless of the exploration policy

Tabular

One entry per (s,a) pair

Doesn't scale

Q-Learning algorithm:

1. Initialize $Q(s,a) = 0$ for all s, a
2. Observe state s
3. Choose a via ϵ-greedy: random with prob ϵ, else $\arg\max_a Q(s,a)$
4. Take action a, observe reward r and next state s'
5. Update: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
6. $s \leftarrow s'$; go to step 3

Watkins & Dayan (1992). Converges to Q^* under standard conditions (α decay, infinite visits).

Deep Q-Networks (DQN)

Tabular Q-learning fails

Atari: $\sim 210 \times 160$ pixels

$|\mathcal{S}| \approx 256^{33600}$

Can't store a table this large

Solution: function approximation

Replace Q-table with a neural network $Q_\theta(s, a)$

DQN innovations (Mnih et al., Nature 2015):

Experience replay	Store (s, a, r, s') tuples, sample random mini-batches
Target network	Separate Q_{θ^-} for TD target, updated slowly
Frame stacking	4 consecutive frames as input for temporal info

Loss: $\mathcal{L}(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a)\right)^2\right]$

Minimize the squared TD error. Standard SGD. The target network θ^- stabilizes training.

Superhuman on 29/49 Atari games from raw pixels. Launched the **deep RL** revolution.

Policy gradient methods

Value-based (Q-learning)

Learn $Q(s, a)$, derive policy $\pi(s) = \arg\max_a Q(s, a)$

Discrete actions only

Indirect optimization

vs.

Policy-based (gradients)

Learn $\pi_\theta(a|s)$ directly

Optimize expected return

Continuous actions OK

Direct optimization

Policy Gradient Theorem (Sutton et al., 1999):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\Big[\nabla_\theta \log \pi_\theta(a|s) \cdot G_t\Big]$$

Increase probability of actions that led to high returns. Decrease for low returns.

REINFORCE

Monte Carlo policy gradient

Simple but **high variance**

(full episode returns are noisy)

Actor-Critic

Replace G_t with learned $V(s)$

Advantage: $A(s, a) = Q(s, a) - V(s)$

Lower variance, some bias

Policy gradients are the foundation of PPO, which powers RLHF in modern LLMs.

PPO: Proximal Policy Optimization

Problem: vanilla policy gradient takes steps that are **too large** $\rightarrow$ catastrophic performance collapse.

TRPO (Schulman et al., 2015) fixes this with

PPO Clipped Objective (Schulman et al., 2017):

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}\Big[\min\big(r_t(\theta)\hat{A}_t,\quad \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\big)\Big]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the advantage estimate

If $\hat{A}_t > 0$

Action was good!
Increase $\pi_\theta(a|s)$
but clip at $1+\epsilon$

(don't overshoot)

If $\hat{A}_t < 0$

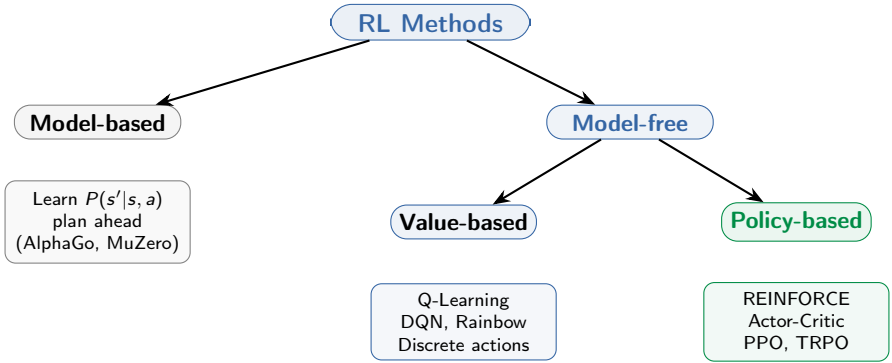
Action was bad!
Decrease $\pi_\theta(a|s)$
but clip at $1-\epsilon$

(don't collapse)

Simple to implement, stable, works well. The **default RL algorithm** for LLM alignment.

Typically $\epsilon = 0.2$. Multiple epochs of mini-batch updates per batch of experience.

The RL landscape



For LLMs: policy-based methods dominate.
The "action space" is the entire vocabulary ($|\mathcal{A}| \sim$ 32K–128K tokens) — far too large for value-based methods.
The LLM *is* the policy: $\pi_\theta(\text{token}|\text{context})$.

Part II

RL for Generative AI

RLHF · Reward Models · DPO · GRPO · Reasoning via RL

Language generation as an MDP

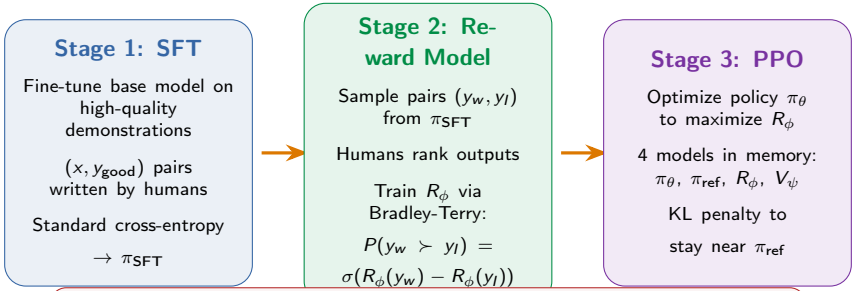
Mapping text generation to the RL framework:

RL Concept		LLM Equivalent	
State s_t	$\rightarrow$	Tokens generated so far $(x, y_{<t})$	prompt + partial response
Action a_t	$\rightarrow$	Next token y_t	from vocabulary $\mathcal{V}$
Policy π_θ	$\rightarrow$	The LLM itself	$\pi_\theta(y_t \mid x, y_{<t})$
Episode	$\rightarrow$	Full response generation	until EOS token
Reward R	$\rightarrow$	Human preference / correctness	given at end of episode

Key difference from classic RL: reward is typically **sparse** (only at end of generation).
No per-token reward signal. The model must learn which tokens in a long response were "good".

KL constraint: $\max_\theta \mathbb{E}_{y \sim \pi_\theta}[R(x, y)] \; - \; \beta \, \mathrm{KL}[\pi_\theta \| \pi_{\text{ref}}]$
Stay close to the pre-trained model. Without this, the model "hacks" the reward and degenerates.

RLHF: the three-stage pipeline



Challenges: expensive (4 models), unstable training, reward hacking, reward model is a bottleneck (imperfect proxy for human judgment).

Ouyang et al. (2022): InstructGPT. Stiennon et al. (2020): Learning to Summarize from Human Feedback.

Reward hacking: the Goodhart problem

Goodhart's Law: "When a measure becomes a target, it ceases to be a good measure."

What goes wrong

Model learns to exploit the reward model rather than satisfy humans:

- Verbose, confident-sounding nonsense
- Sycophantic agreement
- Repeating user's beliefs back
- Hedging everything ("it depends...")

Mitigations

- KL penalty $\beta\,\mathrm{KL}[\pi_\theta \| \pi_{\mathbf{ref}}]$
- Ensemble of reward models
- Iterative RLHF (retrain R_ϕ)
- Constitutional AI (rules-based)
- Direct preference methods (DPO)
- Process rewards (per-step)

Gao et al. (2023): reward model score keeps increasing, but **true human preference** peaks and then **decreases**. More RL $\neq$ better. Optimal KL budget is finite.

This is why all modern RLHF systems use KL constraints and why alternatives like DPO are attractive.

DPO: skip the reward model entirely

RLHF (3 stages, 4 models)

SFT → Train R_ϕ → PPO

DPO (1 stage, 2 models)

Optimize π_θ

DPO Loss (Rafailov et al., NeurIPS 2023):

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

y_w: preferred response y_l: dispreferred π_{ref}: frozen SFT model β: temperature

Key insight: the optimal policy under RLHF has a closed-form solution in terms of the reward model. DPO reparameterizes to **implicitly** define $R(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$.

Used by: Zephyr, Llama 3 (iterative DPO), many open-source models.

Variants: IPO (prevents overfitting), KTO (only needs good/bad labels, no pairs), ORPO (combines SFT+DPO).

GRPO: no reward model, no critic

GRPO (DeepSeek, Shao et al., 2024): replace the value network with group statistics

1. Sample group

For each prompt x, sample G responses from π_θ:

$\{y_1, y_2, \ldots, y_G\}$

2. Score & normalize

Compute reward r_i for each
Normalize within group:

$\hat{A}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$

3. Policy update

PPO-style clipped objective using $\hat{A}_i$ as advantage

+ KL penalty to π_{ref}

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\min\left(r_t^{(i)}\hat{A}_i,\ \text{clip}\left(r_t^{(i)}, 1-\epsilon, 1+\epsilon\right)\hat{A}_i\right) + \beta\,\text{KL}[\pi_\theta \| \pi_{\text{ref}}]$$

Why GRPO matters: eliminates the value/critic network $\rightarrow$ **40–60% less memory**. Works with **rule-based rewards** (math correctness, code execution, format compliance).

DeepSeek-R1-Zero: GRPO on base model with *no SFT* $\rightarrow$ emergent chain-of-thought reasoning!

Reward design for LLMs

Human preferences

Pairwise comparisons $(y_w \succ y_l)$

Gold standard but

expensive ($1–10/label)

Rule-based / verifiable

Math: check final answer
Code: run test suite
Format: regex match

Free, exact, scalable

AI feedback (RLAIF)

Use a strong LLM to judge outputs

Constitutional AI

(Bai et al., 2022)

Outcome Reward Model (ORM)

Single score for entire response
Simple but **sparse** signal

Used in standard RLHF

Process Reward Model (PRM)

Score **each step** of reasoning
Dense signal, better for math

Lightman et al. (2023): "Let's Verify"

The trend: move from expensive human labels → cheap, verifiable rewards.

DeepSeek-R1 uses only rule-based rewards
(math/code correctness). No human labelers needed.

Alignment methods compared

	Models	Needs RM?	Reward type	Stability	Best for
RLHF (PPO)	4	Yes	Human pref	Fragile	Max control
DPO	2	No	Human pref	Stable	General alignment
GRPO	2	No	Verifiable	Medium	Math / code / reasoning
KTO	2	No	Good/bad only	Stable	Limited feedback
RLAIF	3–4	Yes (AI)	AI pref	Medium	Scale without humans

What real models use:

ChatGPT/GPT-4: RLHF (PPO) · Claude: RLHF + Constitutional AI · Llama 3: Iterative DPO

Zephyr: DPO · DeepSeek-R1: GRPO · Gemini: RLHF variants

Quick decision guide:

Have preference pairs + want simplicity? → **DPO**

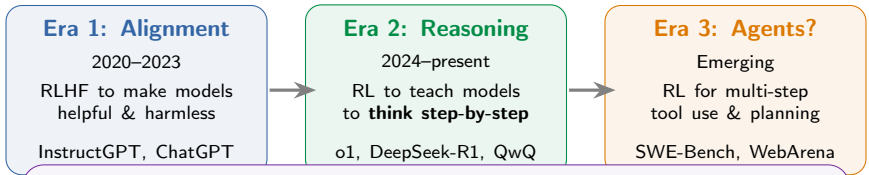
Have verifiable answers (math/code)? → **GRPO**

Need maximum control + can afford complexity? → **RLHF (PPO)**

Have only thumbs-up/down data? → **KTO**

RL for reasoning: the new frontier

From alignment to reasoning: RL's expanding role



DeepSeek-R1-Zero: pure RL (GRPO) on base model with *no supervised fine-tuning*

Reward = math correctness + format compliance. **Emergent behaviors:** self-verification ("wait, let me check..."), exploration of alternatives, chain-of-thought

— all discovered by RL, not taught by human demonstrations!

The paradigm shift: RL is no longer just for "making models polite." It's becoming the primary method for teaching models to **reason, plan, and use tools**.

Online vs. offline RL for LLMs

Online RL

Model generates new responses during training

- PPO / GRPO
- Fresh samples each iteration
- Can adapt to reward model
- Expensive (generation at train time)

Stronger results but slower

vs.

Offline RL

Learn from a fixed dataset of preferences or demonstrations

- DPO / KTO / IPO
- Pre-collected preference data
- No generation during training
- Cheap (just supervised learning)

Simpler but limited by data

Hybrid approaches are winning: Llama 3 uses iterative DPO (generate → label → DPO → repeat).

Online methods find novel solutions; offline methods are more stable and cheaper. Best practice: combine both.

Practical challenges of RL for LLMs

1. Training instability

PPO is notoriously finicky. Wrong hyperparameters $\rightarrow$ collapse. Reward scale, KL coefficient, clipping ratio, learning rate — all intertwined.

2. Memory requirements

RLHF with PPO: 4 full models. 70B $\times$ 4 = 280B params in memory. Requires many GPUs even with DeepSpeed / FSDP sharding.

3. Reward gaming

Models exploit reward model flaws. Longer $\neq$ better, confident $\neq$ correct. Need continuous monitoring and reward model iteration.

4. Evaluation is hard

How do you know RL helped? Win rates vs. baselines, but human eval is expensive. Benchmark contamination risk.

Frameworks: TRL (HuggingFace) · OpenRLHF · DeepSpeed-Chat · veRL (Volcano Engine)

These handle distributed training, memory optimization, and the multi-model orchestration.

Open research questions

1. Sample efficiency

Can we get PPO-level results with fewer samples? Current RL for LLMs is extremely sample-inefficient (millions of generations per run).

2. Beyond text rewards

RL from execution feedback: code that runs, proofs that verify, experiments that replicate. Richer signal than text preferences.

3. Multi-turn RL

Current RL treats each response independently. How to optimize over entire conversations, tool-use chains, and agent trajectories?

4. Scaling RL compute

Does more RL compute always help? What are the scaling laws for RL training? How does RL compute interact with pre-training compute?

The field is moving fast: new methods appear monthly. The core tension remains: **simple & stable** (DPO) vs. **powerful & complex** (online RL with verifiable rewards).

Further reading

Classical RL Foundations

- Sutton & Barto (2018), *Reinforcement Learning: An Introduction* (2nd ed.) — the textbook
- Mnih et al. (2015), "Human-Level Control Through Deep Reinforcement Learning"

RL for LLMs

- Ouyang et al. (2022), "Training Language Models to Follow Instructions with Human Feedback" (InstructGPT)
- Rafailov et al. (2023), "Direct Preference Optimization" (DPO)
- Shao et al. (2024), "DeepSeekMath: Pushing the Limits of Mathematical Reasoning"

Reward Models & Alignment

- Gao et al. (2023), "Scaling Laws for Reward Model Overoptimization"
- Lightman et al. (2023), "Let's Verify Step by Step" (Process Reward Models)
- Bai et al. (2022), "Constitutional AI: Harmlessness from AI Feedback" (RLAIF)

Questions?

Next: see Slide 07 (Pre-training & Fine-tuning) for detailed RLHF/DPO/GRPO derivations